

Introducción a Python para Ingeniería, Datos y Economía

John Leal

2026-09-22

Tabla de contenidos

Prólogo	5
Cómo usar este libro	5
1 Introducción a Python y Google Colab	6
1.1 ¿Qué es Python y por qué lo usaremos en Google Colab?	6
1.2 Tu primer programa	6
1.3 Python como calculadora	7
1.4 Los errores son tus amigos	8
1.5 Guardar tu trabajo	8
1.6 Ejercicios	9
1.7 Resumen	9
2 Fundamentos: variables y tipos de datos	10
2.1 ¿Qué es una variable?	10
2.2 Reglas para nombrar variables	10
2.3 Los cuatro tipos de datos básicos	11
2.4 Convertir entre tipos	12
2.5 Pedir datos al usuario: <code>input()</code>	13
2.6 Imprimir con formato: las f-strings	13
2.7 Comentarios: notas para humanos	14
2.8 Ejercicios	14
2.9 Resumen	15
3 Control de flujo: decisiones, repeticiones y funciones	16
3.1 Decisiones: <code>if</code> , <code>elif</code> , <code>else</code>	16
3.1.1 Operadores de comparación y lógicos	17
3.2 Repeticiones: el ciclo <code>for</code>	18
3.3 Repeticiones con condición: el ciclo <code>while</code>	19
3.4 Empaquetar: las funciones	20
3.5 Ejercicios	21
3.6 Resumen	21
4 Estructuras de datos: listas, tuplas y diccionarios	23
4.1 Listas: colecciones ordenadas que puedes modificar	23
4.1.1 Acceder a un elemento: los índices empiezan en cero	23
4.1.2 Modificar, agregar y contar	24

4.1.3	Operaciones útiles sin escribir ciclos	24
4.1.4	Rebanadas (slicing): tomar una parte	24
4.1.5	Recorrer una lista	25
4.2	Tuplas: colecciones que no deben cambiar	25
4.3	Diccionarios: datos con nombre, no con posición	26
4.3.1	Un diccionario puede guardar una ficha completa	27
4.4	¿Cuál estructura usar?	27
4.5	Ejercicios	28
4.6	Resumen	28
5	NumPy y gráficas: cómputo científico	29
5.1	Librerías: código que otros escribieron por ti	29
5.2	El array: una lista con superpoderes	29
5.3	Crear secuencias de datos	30
5.4	Estadística instantánea	30
5.5	Gráficas con Matplotlib	31
5.5.1	Otros tipos de gráfica útiles	32
5.6	Aplicación de ingeniería: simular y graficar un fenómeno	35
5.7	Ejercicios	36
5.8	Resumen	37
6	pandas: manejo de datos	38
6.1	El DataFrame: una tabla con superpoderes	38
6.2	Inspeccionar los datos	39
6.3	Seleccionar y filtrar	40
6.4	Columnas calculadas	40
6.5	Agrupar: la pregunta de negocio típica	41
6.6	Leer datos reales: <code>read_csv</code>	41
6.7	Series de tiempo (primera mirada)	43
6.8	Ejercicios	44
6.9	Resumen	44
7	Aplicaciones en administración y economía	46
7.1	Valor futuro y valor presente	46
7.2	Valor Actual Neto (VAN): ¿vale la pena el proyecto?	47
7.3	Tasa Interna de Retorno (TIR)	47
7.4	Regresión lineal: aprender de los datos	48
7.5	Simulación Monte Carlo: decidir con incertidumbre	49
7.6	Ejercicios	51
7.7	Resumen	52
8	Aplicaciones en ingeniería: agro, mecatrónica, veterinaria y zootecnia	53
8.1	Ingeniería agrícola: ¿cuánta agua necesita el cultivo?	53

8.2	Mecatrónica: limpiar la señal de un sensor	55
8.3	Veterinaria: dosificación por peso vivo	56
8.4	Zootecnia: curva de crecimiento y ganancia diaria	57
8.5	Ejercicios	58
8.6	Resumen	59
9	Proyectos integradores	60
9.1	Proyecto A — Ingeniería agrícola: ¿cuándo y cuánto regar?	60
9.2	Proyecto B — Mecatrónica: confiabilidad de un sensor de bajo costo	61
9.3	Proyecto C — Administración y economía: evaluación completa de un proyecto productivo	61
9.4	Proyecto D — Veterinaria y zootecnia: plan de ceba de un lote	62
9.5	Proyecto E — Ciencia de datos general: auditoría de un dataset real	63
9.6	Rúbrica sugerida (para docentes)	63
10	Capítulo 8: Introducción a big data	64

Prólogo

Este libro está pensado para estudiantes y profesionales de ingeniería, análisis de datos y economía que desean aprender a programar en Python desde cero. No se asume ninguna experiencia previa en programación: los conceptos se introducen de forma progresiva, con ejemplos cercanos a la ingeniería, la estadística y la economía, para que cada tema tenga un contexto claro y aplicable.

Todo el código está diseñado para ejecutarse en Google Colab, el entorno gratuito de notebooks de Google. No necesitas instalar nada en tu computadora: basta un navegador y una cuenta de Google. Cada capítulo contiene bloques de código listos para copiar y pegar en una celda de Colab, de modo que puedes leer, ejecutar y experimentar de inmediato.

Al finalizar el libro serás capaz de escribir programas en Python, procesar datos con pandas, resolver problemas de ingeniería y economía (valor futuro, VAN, simulación Monte Carlo) y entender los conceptos básicos del mundo del big data.

Cómo usar este libro

1. **Lee con Colab abierto.** Cada bloque de código está hecho para copiarse y pegarse directamente en una celda de Colab. Ejecuta todo lo que veas: programar se aprende haciendo.
2. **Modifica los ejemplos.** Cambia los valores, rompe el código a propósito, lee los errores. Experimentar es parte del método.
3. **Haz los ejercicios.** Al final de cada capítulo hay ejercicios y un mini-reto que integra lo aprendido con un problema de ingeniería, administración o economía.
4. **Lleva un notebook por capítulo.** Serán tu cuaderno de apuntes ejecutable y tu portafolio de práctica.

La ruta del libro es progresiva: los capítulos 1 a 4 construyen los fundamentos del lenguaje; el 5 y el 6 introducen el cómputo científico y el manejo de datos; el 7 aplica todo a problemas de administración y economía; y el 8 cierra con una introducción a big data.

1 Introducción a Python y Google Colab

1.1. ¿Qué es Python y por qué lo usaremos en Google Colab?

Python es un lenguaje de programación de propósito general, creado con una idea central: que el código se lea casi como texto en inglés. Esa claridad lo convirtió en el lenguaje más usado del mundo en ciencia de datos, ingeniería, economía e inteligencia artificial.

¿Por qué Python y no otro lenguaje? Porque con las mismas diez líneas de código que en otros lenguajes apenas configurarían el entorno, en Python ya estás analizando datos reales, graficando una función o calculando el valor presente neto de un proyecto.

En este libro trabajaremos con **Google Colab**: un entorno gratuito de Google que corre en el navegador. No necesitas instalar nada en tu computador; solo una cuenta de Google. Cada bloque de código de este libro está diseñado para que lo copies, lo pegues en una celda de Colab y lo ejecutes.

1.2. Tu primer programa

1. Abre colab.research.google.com en tu navegador.
2. Haz clic en **Nuevo notebook**.
3. En la primera celda, escribe (o copia) lo siguiente:

```
print("Hola, mundo")
```

```
Hola, mundo
```

4. Ejecuta la celda con el botón  o con las teclas **Shift + Enter**.

La caja oscura debajo del código muestra la **salida**: lo que el programa imprime. En este libro verás siempre ese par — el código y su resultado — igual que en Colab.

¿Qué acaba de pasar? `print()` es una **función**: una instrucción que le dice a Python “muestra esto en pantalla”. El texto entre comillas se llama **cadena de caracteres** (*string*). Acabas de escribir y ejecutar tu primer programa.

Nota

Un notebook de Colab tiene dos tipos de celdas: **código** (se ejecuta) y **texto** (se lee, como este párrafo). Agrégalas con los botones + Código y + Texto.

1.3. Python como calculadora

Antes de programar “en serio”, usa Python como una calculadora. Copia y ejecuta cada línea:

```
2 + 3
```

```
5
```

```
10 / 4
```

```
2.5
```

```
2 ** 10
```

```
1024
```

```
17 % 5
```

```
2
```

Los operadores básicos son:

Tabla 1.1: Operadores aritméticos básicos de Python

Operador	Significado	Ejemplo	Resultado
+	Suma	2 + 3	5
-	Resta	7 - 4	3
*	Multiplicación	3 * 5	15
/	División	10 / 4	2.5
**	Potencia	2 ** 10	1024
//	División entera	17 // 5	3

Operador	Significado	Ejemplo	Resultado
%	Residuo (módulo)	17 % 5	2

Un ejemplo con sentido económico: si inviertes \$1.000.000 a una tasa del 12% anual durante 5 años, el valor futuro es:

```
1000000 * (1 + 0.12) ** 5
```

```
1762341.6832000008
```

Observa que Python respeta el orden de las operaciones (primero paréntesis, luego potencias, luego multiplicación y división, al final suma y resta), igual que en álgebra.

1.4. Los errores son tus amigos

Escribe esto en una celda y ejecútalo:

```
print("Hola"
```

Verás algo como:

```
SyntaxError: incomplete input
```

No te alarmes: un error no rompe nada. Python te está diciendo **qué** falló (**SyntaxError**: error de sintaxis) y en este caso es evidente: falta cerrar el paréntesis. Corrige y ejecuta de nuevo.



Tip

Regla de oro para todo el libro: **lee los errores de abajo hacia arriba**. La última línea del mensaje casi siempre dice exactamente qué salió mal.

1.5. Guardar tu trabajo

Colab guarda tus notebooks automáticamente en tu Google Drive (carpeta **Colab Notebooks**). Buenas prácticas desde el primer día:

- Renombra el notebook: clic en el título de arriba a la izquierda. Usa nombres sin espacios ni tildes: `capitulo01_introduccion.ipynb`.
- Para descargarlo: Archivo → Descargar → Descargar `.ipynb`.
- Crea un notebook por capítulo de este libro; te servirán como tu propio cuaderno de apuntes ejecutable.

1.6. Ejercicios

1. Ejecuta `print("Tu nombre")` con tu nombre real.
2. Calcula cuántos segundos tiene un año (365 días) usando solo operadores.
3. Calcula el valor futuro de \$5.000.000 al 8 % anual durante 10 años.
4. Provoca a propósito un error quitando una comilla en un `print`, lee el mensaje y corrígelo.
5. **Mini-reto:** una hectárea produce 4,2 toneladas de café. ¿Cuántas toneladas producen 37 hectáreas? ¿Y si el precio es \$9.500.000 por tonelada, cuánto vale la cosecha total? Resuélvelo en una sola celda con dos operaciones.

1.7. Resumen

- Python es un lenguaje claro y es el estándar en datos e ingeniería.
- Colab te permite programar sin instalar nada, desde el navegador.
- `print()` muestra resultados en pantalla.
- Python funciona como calculadora con los operadores `+` `-` `*` `/` `**` `//` `%`.
- Los errores son mensajes de ayuda: léelos de abajo hacia arriba.

2 Fundamentos: variables y tipos de datos

2.1. ¿Qué es una variable?

Imagina una caja con una etiqueta. Dentro de la caja guardas algo (un número, un texto), y la etiqueta te permite encontrarla después. Eso es una **variable**: un nombre que guarda un valor.

En Python, guardar un valor en una variable se llama **asignar**, y se hace con el signo `=`:

```
precio = 9500
```

Lee esa línea así: “la variable `precio` guarda el valor 9500”. Ojo: el `=` de Python **no es** el igual de las matemáticas; no afirma que dos cosas sean iguales, sino que **guarda** el valor de la derecha en el nombre de la izquierda.

A partir de ahí, puedes usar la variable en cualquier cálculo:

```
precio = 9500
cantidad = 37
total = precio * cantidad
print(total)
```

```
351500
```

La gran ventaja: si el precio cambia, solo modificas una línea y todo lo demás se recalcula solo. Cambia `precio` a 10200 y vuelve a ejecutar.

2.2. Reglas para nombrar variables

Python es estricto con los nombres. Estos son válidos:

```
precio_tonelada = 9500
temperatura2 = 25.4
nombre_finca = "La Esperanza"
```

Estos **no** son válidos (ejecútalos en Colab y lee el error):

```
2temperatura = 25      # no puede empezar con número
precio-tonelada = 9500 # no puede tener guiones
precio tonelada = 9500 # no puede tener espacios
```

Nota

Convención de este libro: nombres en minúsculas, sin tildes, con palabras separadas por guion bajo: `valor_futuro`, `tasa_anual`, `numero_hectareas`. No es obligatorio, pero es el estilo que usa todo el mundo de Python.

2.3. Los cuatro tipos de datos básicos

Todo valor en Python tiene un **tipo**. Para empezar solo necesitas conocer cuatro:

Tipo	Nombre	Qué guarda	Ejemplo
int	Entero	Números sin decimales	37, -5, 1000000
float	Decimal	Números con decimales	4.2, 0.12, -3.5
str	Texto	Cadenas de caracteres	"café", "La Esperanza"
bool	Booleano	Verdadero o falso	True, False

Python detecta el tipo automáticamente según lo que guardes. Puedes preguntárselo con la función `type()`:

```
precio = 9500
print(type(precio))
```

```
<class 'int'>
```

```
rendimiento = 4.2
print(type(rendimiento))
```

```
<class 'float'>
```

```
nombre = "Ana"
print(type(nombre))
```

```
<class 'str'>
```

```
es_rentable = True
print(type(es_rentable))
```

```
<class 'bool'>
```

Advertencia

Cuidado con el punto y la coma: en Python los decimales se escriben con **punto**, no con coma. Escribe 4.2, no 4,2. Si escribes 4,2 Python no dará un error obvio: pensará que son dos números separados. Es uno de los errores más comunes de quienes empiezan en países hispanos.

2.4. Convertir entre tipos

A veces necesitas transformar un tipo en otro. Se hace con funciones que tienen el nombre del tipo: `int()`, `float()`, `str()`.

El caso clásico: los números que vienen como texto no se pueden sumar:

```
a = "2"
b = "3"
print(a + b)
```

```
23
```

¿Sorprendido? El resultado es "23", no 5. Como `a` y `b` son **texto**, el `+` los une en lugar de sumarlos (pegar textos se llama **concatenar**). La solución es convertirlos primero:

```
a = "2"
b = "3"
print(int(a) + int(b))
```

```
5
```

2.5. Pedir datos al usuario: `input()`

Hasta ahora los valores estaban escritos en el código. Pero normalmente querrás preguntarle a la persona que usa tu programa. Eso se hace con `input()`:

```
nombre = input("¿Cómo te llamas? ")
print("Hola,", nombre)
```

En Colab verías algo como:

```
¿Cómo te llamas? Ana
Hola, Ana
```

Al ejecutar, el programa se detiene, muestra la pregunta y espera a que escribas algo y presiones **Enter**.

! Importante

Lo que `input()` devuelve siempre es texto (`str`), aunque la persona escriba un número. Si vas a hacer cuentas con esa respuesta, conviértela primero con `int()` o `float()`.

```
edad = input("¿Cuántos años tienes? ")
edad = int(edad)
print("El próximo año tendrás", edad + 1)
```

O en una sola línea, que es como lo verás en la práctica:

```
edad = int(input("¿Cuántos años tienes? "))
```

2.6. Imprimir con formato: las f-strings

Ya sabes que `print()` muestra cosas. Cuando quieras mezclar texto con variables de forma elegante, usa una **f-string**: una cadena con una `f` antes de las comillas, donde las variables van entre llaves `{}`:

```
precio = 9500
cantidad = 37
total = precio * cantidad
print(f"El total de la cosecha es ${total}")
```

```
El total de la cosecha es $351500
```

Todo lo que está dentro de las llaves se evalúa. Puedes incluso hacer cuentas ahí dentro:

```
tasa = 0.12
capital = 1000000
print(f"En 5 años tendrás ${capital * (1 + tasa) ** 5}")
```

```
En 5 años tendrás $1762341.6832000008
```

2.7. Comentarios: notas para humanos

Todo lo que escribas después de un `#` en una línea, Python lo ignora por completo. Sirve para dejar notas que expliquen tu código:

```
# Cálculo del valor futuro a 5 años
tasa = 0.12          # tasa anual
capital = 1000000    # inversión inicial en pesos
vf = capital * (1 + tasa) ** 5
print(f"Valor futuro: ${vf}")
```

```
Valor futuro: $1762341.6832000008
```

Comenta tu código pensando en la persona que lo leerá en seis meses. Esa persona, casi siempre, serás tú.

2.8. Ejercicios

1. Crea tres variables con tu nombre, tu edad y tu estatura (en metros, con decimal). Imprímelas con una f-string en una sola frase.
2. Pregunta al usuario cuántas hectáreas tiene su finca y cuántas toneladas produce por hectárea. Calcula e imprime la producción total.
3. Pregunta al usuario su salario mensual y calcula cuánto gana al año. Imprime el resultado con una f-string.

4. Ejecuta esto en Colab, observa qué pasa y corrígelo:

```
precio = input("Precio: ")
print(precio * 2)
```

(Pista: multiplicar texto por dos no es lo que queremos).

5. **Mini-reto (economía):** pide al usuario un capital inicial, una tasa de interés anual (en porcentaje, ej. 12) y un número de años. Calcula el valor futuro con interés compuesto e imprímelo con formato: **Con \$X al Y% durante Z años, tendrás \$W**. Recuerda convertir la tasa de porcentaje a decimal dividiendo entre 100.

2.9. Resumen

- Una **variable** es un nombre que guarda un valor; se asigna con `=`.
- Los cuatro tipos básicos: `int` (enteros), `float` (decimales), `str` (texto), `bool` (verdadero/falso).
- Los decimales se escriben con **punto**: 4.2.
- `input()` siempre devuelve texto; usa `int()` o `float()` para hacer cuentas.
- Las **f-strings** (`f"...{variable}..."`) mezclan texto y valores fácilmente.
- Los **comentarios** con `#` documentan tu código.

3 Control de flujo: decisiones, repeticiones y funciones

Hasta ahora tus programas ejecutan las líneas una tras otra, de arriba hacia abajo, siempre igual. Los programas útiles hacen tres cosas más: **deciden** (si pasa esto, haz aquello), **repiten** (haz esto 1000 veces) y **empaquetan** (guarda este procedimiento para usarlo cuando quiera). Eso es el control de flujo.

3.1. Decisiones: if, elif, else

Un sistema de riego automático tiene una lógica muy simple: *si la humedad del suelo es baja, activar la bomba*. En Python eso se escribe casi igual que en español:

```
humedad = 25 # porcentaje medido por el sensor

if humedad < 30:
    print("Humedad baja: activar riego")
```

```
Humedad baja: activar riego
```

Ahora cambia humedad a 45 y vuelve a ejecutar: no aparece nada, porque la condición ya no se cumple.

Para cubrir más casos se usan **elif** (“si no, entonces si...”) y **else** (“en cualquier otro caso”):

```
humedad = 25

if humedad < 30:
    print("Humedad baja: activar riego")
elif humedad < 60:
    print("Humedad adecuada: no hacer nada")
else:
    print("Humedad excesiva: revisar drenaje")
```

```
Humedad baja: activar riego
```

⚠ Advertencia

La indentación manda. Fíjate que las líneas después de `if` van desplazadas 4 espacios a la derecha. Ese sangrado le dice a Python qué pertenece a cada bloque. Si lo quitas, da error. Es la regla más famosa de Python: la sangría no es decoración, es sintaxis.

3.1.1. Operadores de comparación y lógicos

Las condiciones se construyen con estos operadores:

Tabla 3.1: Operadores de comparación y lógicos

Tipo	Operador	Significado	Ejemplo	Resultado
Comparación	<code>==</code>	Igual a	<code>5 == 5</code>	<code>True</code>
Comparación	<code>!=</code>	Diferente de	<code>5 != 3</code>	<code>True</code>
Comparación	<code><</code>	Menor que	<code>25 < 30</code>	<code>True</code>
Comparación	<code>></code>	Mayor que	<code>7 > 4</code>	<code>True</code>
Comparación	<code><=</code>	Menor o igual	<code>3 <= 2</code>	<code>False</code>
Comparación	<code>>=</code>	Mayor o igual	<code>30 >= 30</code>	<code>True</code>
Lógico	<code>and</code>	Y (ambas verdaderas)	<code>True and False</code>	<code>False</code>
Lógico	<code>or</code>	O (al menos una)	<code>True or False</code>	<code>True</code>
Lógico	<code>not</code>	Negación	<code>not True</code>	<code>False</code>

! Importante

Doble igual para comparar. Es el error número 1 de los principiantes: `=` asigna, `==` compara. `if humedad = 30:` es error; `if humedad == 30:` es correcto.

Con `and` y `or` puedes combinar condiciones, como una alarma de invernadero que se activa por temperatura **o** por humedad:

```
temperatura = 38
humedad = 45

if temperatura > 35 or humedad > 80:
    print("ALARMA: condiciones fuera de rango")
else:
```

```
print("Condiciones normales")
```

```
ALARMA: condiciones fuera de rango
```

3.2. Repeticiones: el ciclo for

Un sensor toma una lectura cada hora. Un administrador calcula la nómina de 50 empleados. Un ingeniero evalúa 12 parcelas. En todos los casos: **la misma operación, muchas veces**. Para eso está el ciclo `for`.

La forma más simple usa `range()`, que genera una secuencia de números:

```
for hora in range(6):  
    print(f"Tomando lectura de la hora {hora}")
```

```
Tomando lectura de la hora 0  
Tomando lectura de la hora 1  
Tomando lectura de la hora 2  
Tomando lectura de la hora 3  
Tomando lectura de la hora 4  
Tomando lectura de la hora 5
```

Esto ejecuta el `print` 6 veces, con `hora` tomando los valores 0, 1, 2, 3, 4, 5. (Usamos 6 para no llenar la página; en la práctica serían 24 lecturas al día).

Ejemplo agro: calcular la producción total recorriendo las parcelas de una finca:

```
producciones = [4.2, 3.8, 5.1, 4.6, 3.9] # toneladas por parcela  
total = 0  
  
for p in producciones:  
    total = total + p  
  
print(f"Producción total: {total} toneladas")
```

```
Producción total: 21.599999999999998 toneladas
```

El patrón `total = total + p` se llama **acumulador**: una variable que empieza en cero y va sumando. Lo usarás constantemente.

Ejemplo de administración: proyectar las ventas de los próximos 5 años con crecimiento del 8% anual:

```
ventas = 50000000 # ventas del año actual

for año in range(1, 6):
    ventas = ventas * 1.08
    print(f"Año {año}: ${ventas:,.0f}")
```

```
Año 1: $54,000,000
Año 2: $58,320,000
Año 3: $62,985,600
Año 4: $68,024,448
Año 5: $73,466,404
```

Fíjate en el truco `{ventas:,.0f}` dentro de la f-string: formatea el número con separador de miles y sin decimales.

3.3. Repeticiones con condición: el ciclo `while`

El `for` repite un número conocido de veces. El `while` repite **mientras una condición sea verdadera** — cuando no sabes cuántas vueltas harán falta:

```
tanque = 0 # litros
capacidad = 1000

while tanque < capacidad:
    tanque = tanque + 150 # la bomba llena 150 litros por ciclo
    print(f"Nivel actual: {tanque} litros")

print("Tanque lleno")
```

```
Nivel actual: 150 litros
Nivel actual: 300 litros
Nivel actual: 450 litros
Nivel actual: 600 litros
Nivel actual: 750 litros
Nivel actual: 900 litros
Nivel actual: 1050 litros
```

Tanque lleno

⚠ Advertencia

Si la condición de un `while` jamás se hace falsa, el ciclo corre para siempre (ciclo infinito). En Colab lo detienes con el botón . Asegúrate siempre de que algo dentro del ciclo acerque la condición a su fin.

3.4. Empaquetar: las funciones

Ya usaste funciones que vienen con Python: `print()`, `input()`, `int()`. Ahora crearás las tuyas. Una función es un procedimiento con nombre: le das entradas (**parámetros**), hace su trabajo y te devuelve un resultado (**retorno**).

```
def valor_futuro(capital, tasa, años):  
    vf = capital * (1 + tasa) ** años  
    return vf
```

- `def` le dice a Python “voy a definir una función”.
- Entre paréntesis van los parámetros: lo que la función necesita recibir.
- `return` entrega el resultado y termina la función.

Se usa así, todas las veces que quieras:

```
inversion1 = valor_futuro(1000000, 0.12, 5)  
inversion2 = valor_futuro(5000000, 0.08, 10)  
  
print(f"Proyecto A: ${inversion1:,.0f}")  
print(f"Proyecto B: ${inversion2:,.0f}")
```

```
Proyecto A: $1,762,342  
Proyecto B: $10,794,625
```

La ventaja enorme: la fórmula se escribe **una sola vez**. Si mañana hay que corregirla, se corrige en un solo lugar.

Otro ejemplo, ahora de mecatrónica: una función que clasifica la lectura de un sensor de temperatura:

```
def clasificar_temperatura(t):
    if t < 15:
        return "frío"
    elif t <= 30:
        return "óptimo"
    else:
        return "caliente"

print(clasificar_temperatura(22))
print(clasificar_temperatura(38))
```

```
óptimo
caliente
```

3.5. Ejercicios

1. Escribe un `if` que imprima “Crédito aprobado” si el puntaje crediticio es mayor o igual a 700, y “Crédito rechazado” en caso contrario. Prueba con 650 y con 720.
2. Un motor debe apagarse si la temperatura supera 90 °C y el voltaje es inestable (variable booleana `voltaje_inestable`). Escribe la condición con `and` y pruébala con distintos valores.
3. Con un `for` y un acumulador, suma los costos de una lista de insumos: [120000, 85000, 210000, 45000]. Imprime el total.
4. Escribe una función `area_parcela(largo, ancho)` que devuelva el área en metros cuadrados, y úsala para tres parcelas distintas.
5. Escribe una función `es_rentable(ingresos, costos)` que devuelva `True` si los ingresos superan a los costos y `False` si no.
6. **Mini-reto (agro + economía):** tienes las producciones de 6 parcelas: [4.2, 2.8, 5.1, 3.3, 4.6, 2.1] toneladas. Con un `for` y un contador, calcula cuántas parcelas produjeron menos de 3 toneladas (bajo rendimiento) e imprime el porcentaje que representan del total. Luego empaqueta tu solución en una función `porcentaje_bajo_rendimiento(producciones, umbral)`.

3.6. Resumen

- `if` / `elif` / `else` ejecutan bloques según condiciones; la **indentación define los bloques**.
- Comparar con `==`, asignar con `=`. Combinar condiciones con `and`, `or`, `not`.

- **for** repite un número conocido de veces; **while** repite mientras la condición sea verdadera.
- El **acumulador** (`total = total + x`) es el patrón básico para sumar en ciclos.
- Las **funciones** (`def ... return`) empaquetan procedimientos para reutilizarlos sin duplicar código.

4 Estructuras de datos: listas, tuplas y diccionarios

Una variable guarda **un** valor. Pero en la práctica manejas colecciones: las lecturas de un sensor durante el día, los precios de 20 productos, las coordenadas de las parcelas. Para eso existen las estructuras de datos. En este capítulo verás las tres esenciales: **listas**, **tuplas** y **diccionarios**.

4.1. Listas: colecciones ordenadas que puedes modificar

Una lista es una secuencia de valores entre corchetes [], separados por comas:

```
lecturas = [22.5, 23.1, 24.0, 23.8, 25.2] # temperaturas de un sensor
print(lecturas)
```

```
[22.5, 23.1, 24.0, 23.8, 25.2]
```

4.1.1. Acceder a un elemento: los índices empiezan en cero

Cada elemento tiene una posición llamada **índice**. La regla que confunde a todo principiante: **el primer elemento es el índice 0**, no el 1:

```
lecturas = [22.5, 23.1, 24.0, 23.8, 25.2]

print(lecturas[0]) # primera lectura
print(lecturas[2]) # tercera lectura
print(lecturas[-1]) # última lectura (el -1 cuenta desde el final)
```

```
22.5
24.0
25.2
```

4.1.2. Modificar, agregar y contar

Las listas son **mutables**: puedes cambiarlas después de crearlas:

```
precios = [9500, 8200, 10500]

precios[0] = 9800      # modificar: el primer precio cambió
precios.append(7600)   # agregar al final
print(precios)
print(len(precios))    # cuántos elementos tiene
```

```
[9800, 8200, 10500, 7600]
4
```

4.1.3. Operaciones útiles sin escribir ciclos

Python trae funciones que resumen una lista en una línea:

```
producciones = [4.2, 3.8, 5.1, 4.6, 3.9]

print(sum(producciones))    # total
print(max(producciones))    # la mayor
print(min(producciones))    # la menor
print(len(producciones))    # cuántas parcelas
print(sum(producciones) / len(producciones)) # promedio
```

```
21.599999999999998
5.1
3.8
5
4.3199999999999999
```

4.1.4. Rebanadas (slicing): tomar una parte

Con `[inicio:fin]` tomas un pedazo de la lista (incluye el inicio, **no** incluye el fin):

```
lecturas = [22.5, 23.1, 24.0, 23.8, 25.2]

print(lecturas[1:3])    # del índice 1 al 2
print(lecturas[:2])     # las dos primeras
print(lecturas[3:])     # del índice 3 al final
```

```
[23.1, 24.0]
[22.5, 23.1]
[23.8, 25.2]
```

4.1.5. Recorrer una lista

Ya lo hiciste en el capítulo anterior: el `for` nació para esto:

```
ventas = [12.5, 15.2, 9.8, 18.1] # millones por trimestre

for v in ventas:
    if v < 10:
        print(f"Trimestre bajo: {v}")
```

```
Trimestre bajo: 9.8
```

4.2. Tuplas: colecciones que no deben cambiar

Una tupla se escribe con paréntesis `()` y es **immutable**: una vez creada, no se puede modificar. Sirve para datos que conceptualmente van juntos y no deben alterarse, como las coordenadas de una parcela:

```
ubicacion = (3.42, -76.53) # latitud, longitud

print(ubicacion[0]) # latitud
```

```
3.42
```

Si intentas modificarla, Python te protege con un error:

```
ubicacion[0] = 4.0 # TypeError: las tuplas no se modifican
```

```
TypeError: 'tuple' object does not support item assignment
```

Regla práctica: si los datos representan algo fijo (coordenadas, una fecha, un registro que no debe tocarse), usa tupla. Si van a cambiar (lecturas, precios, inventarios), usa lista.

4.3. Diccionarios: datos con nombre, no con posición

En una lista accedes por posición (`precios[0]`). Pero ¿y si quieres acceder **por nombre**? Para eso están los diccionarios: pares de **clave** y **valor** entre llaves `{}`:

```
precios = {  
    "café": 9500,  
    "cacao": 8200,  
    "banano": 7600  
}  
  
print(precios["café"]) # acceder por clave
```

```
9500
```

Agregar y modificar es asignar a una clave:

```
precios = {"café": 9500, "cacao": 8200, "banano": 7600}  
  
precios["banano"] = 7800 # modificar  
precios["panela"] = 5200 # agregar un producto nuevo  
print(precios)
```

```
{'café': 9500, 'cacao': 8200, 'banano': 7800, 'panela': 5200}
```

Recorrer un diccionario con `.items()` te da clave y valor a la vez:

```
inventario = {"motobombas": 3, "sensores_humedad": 12, "valvulas": 25}  
  
for equipo, cantidad in inventario.items():  
    if cantidad < 5:  
        print(f"Reponer {equipo}: quedan {cantidad}")
```

```
Reponer motobombas: quedan 3
```

4.3.1. Un diccionario puede guardar una ficha completa

Los diccionarios brillan cuando un valor es a su vez otra estructura:

```
empleado = {
    "nombre": "María Pérez",
    "cargo": "Analista",
    "salario": 2800000,
    "areas": ["compras", "logística"]
}

print(empleado["nombre"])
print(empleado["salario"] * 12)    # salario anual
```

```
María Pérez
33600000
```

Este patrón — una ficha con campos nombrados — es la base de casi todo el manejo de datos que verás en el capítulo de pandas.

4.4. ¿Cuál estructura usar?

Estructura	Sintaxis	¿Se modifica?	Acceso por	Úsala para
Lista	[1, 2, 3]	Sí	Posición	Series de mediciones, colecciones que crecen
Tupla	(1, 2)	No	Posición	Datos fijos que van juntos (coordenadas)
Diccionario	{"a": 1}	Sí	Nombre (clave)	Fichas, inventarios, catálogos, configuración

4.5. Ejercicios

1. Crea una lista con las temperaturas de 7 días. Imprime la máxima, la mínima y el promedio (con `sum` y `len`).
2. De la lista `costos = [45000, 120000, 38000, 90000, 67000]`, imprime solo los costos mayores a 50000 usando un `for` y un `if`.
3. Crea una tupla con la coordenada de tu ciudad y otra con la de una finca. Imprime la diferencia de latitud entre ambas.
4. Crea un diccionario con 4 productos agrícolas y sus precios. Sube todos los precios un 10% recorriéndolo con `.items()` y reasignando cada valor.
5. Crea un diccionario que represente un sensor con las claves `id`, `tipo`, `parcela` y `lecturas` (una lista de 5 valores). Imprime el promedio de sus lecturas accediendo a través del diccionario.
6. **Mini-reto (administración):** tienes esta lista de ventas mensuales (millones): `[42, 38, 55, 61, 47, 39, 52, 58, 63, 49, 44, 57]`. Sin usar funciones hechas, solo un `for` y acumuladores: calcula el promedio anual, cuenta cuántos meses superaron el promedio e imprime ambos resultados. Luego, guarda en un diccionario `resumen` las claves `"promedio"` y `"meses_sobre_promedio"`.

4.6. Resumen

- **Lista** `[]`: ordenada y modificable; el caballo de batalla. Índices desde `0`.
- `append()` agrega, `len()` cuenta, `sum()/max()/min()` resumen, `[i:j]` rebana.
- **Tupla** `()`: igual a la lista pero inmutable; para datos fijos.
- **Diccionario** `{}`: pares clave-valor; acceso por nombre con `d["clave"]`, recorrido con `.items()`.
- Elegir bien la estructura es la mitad del programa.

5 NumPy y gráficas: cómputo científico

Hasta aquí trabajaste con valores sueltos y listas. La ingeniería y la economía manejan otra cosa: **miles de datos** a la vez — todas las lecturas de un sensor, los precios de un año, la producción de 500 parcelas. Hacer eso con listas y ciclos `for` es lento y enredado. La solución es **NumPy**: la librería que convierte a Python en una máquina de cálculo numérico. Y para ver los resultados, **Matplotlib**: la librería de gráficas.

5.1. Librerías: código que otros escribieron por ti

Una **librería** es un paquete de funciones listas para usar. Se carga con `import`:

```
import numpy as np
```

La convención `as np` le pone un apodo corto: en vez de escribir `numpy.array` escribes `np.array`. En Colab, NumPy y Matplotlib ya vienen instaladas; solo hay que importarlas.

5.2. El array: una lista con superpoderes

El objeto central de NumPy es el **array**. Se parece a una lista, pero permite operar sobre **todos sus elementos a la vez**:

```
import numpy as np

rendimientos = np.array([4.2, 3.8, 5.1, 4.6, 3.9]) # toneladas por hectárea

# Convertir todo a kg/ha de una sola vez:
rendimientos_kg = rendimientos * 1000
print(rendimientos_kg)
```

```
[4200. 3800. 5100. 4600. 3900.]
```

Con una lista normal, `lista * 1000` daría error (o algo raro). Con un array, la operación se aplica elemento por elemento. Esto se llama **vectorización** y es la idea más importante del capítulo.

Otro ejemplo típico de mecatrónica: un sensor entrega temperaturas en Celsius y el sistema las necesita en Fahrenheit:

```
celsius = np.array([20.0, 22.5, 25.1, 31.8, 28.4])
fahrenheit = celsius * 9 / 5 + 32
print(fahrenheit)
```

```
[68.    72.5   77.18  89.24  83.12]
```

Ni un solo ciclo. La fórmula se escribe como en álgebra y se aplica a todo el arreglo.

5.3. Crear secuencias de datos

NumPy tiene funciones para generar arrays sin escribir los valores a mano:

```
print(np.arange(0, 10, 2))    # de 0 a 10 (sin incluir), de 2 en 2
print(np.linspace(0, 1, 5))  # 5 valores igualmente espaciados entre 0 y 1
print(np.zeros(4))           # cuatro ceros
print(np.ones(3))            # tres unos
```

```
[0 2 4 6 8]
[0.  0.25 0.5  0.75 1.  ]
[0. 0. 0. 0.]
[1. 1. 1.]
```

`linspace` es el favorito para graficar funciones: genera los puntos del eje x.

5.4. Estadística instantánea

Los arrays traen métodos estadísticos incorporados:

```
produccion = np.array([4.2, 3.8, 5.1, 4.6, 3.9, 4.8, 2.9, 5.0])
```

```
print(f"Promedio:      {produccion.mean():.2f} ton/ha")
print(f"Desviación:    {produccion.std():.2f}")
print(f"Máximo:        {produccion.max()} ton/ha")
print(f"Parcela del máximo: {produccion.argmax()}")
```

```
Promedio:      4.29 ton/ha
Desviación:    0.69
Máximo:        5.1 ton/ha
Parcela del máximo: 2
```

`argmax()` no devuelve el valor máximo sino su **posición**: útil para saber *cuál* parcela fue la mejor.

5.5. Gráficas con Matplotlib

Un número dice poco; una gráfica lo dice todo. Matplotlib se importa así:

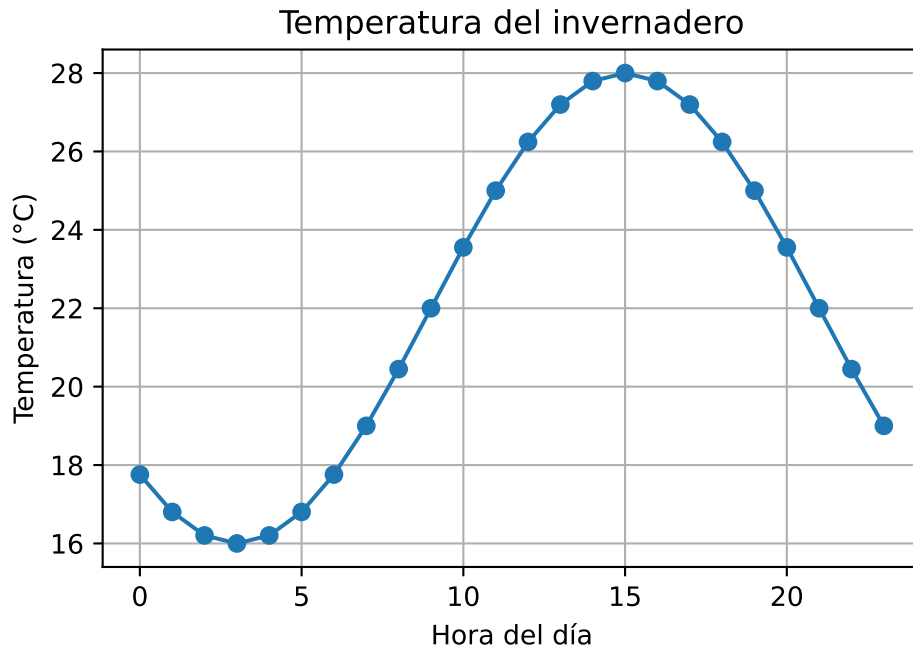
```
import matplotlib.pyplot as plt
```

La gráfica mínima tiene cuatro ingredientes: datos, `plot`, etiquetas y `show`:

```
import numpy as np
import matplotlib.pyplot as plt

horas = np.arange(0, 24) # 0 a 23
temperatura = 22 + 6 * np.sin((horas - 9) * np.pi / 12) # modelo simple de un día

plt.plot(horas, temperatura, marker="o")
plt.xlabel("Hora del día")
plt.ylabel("Temperatura (°C)")
plt.title("Temperatura del invernadero")
plt.grid(True)
plt.show()
```

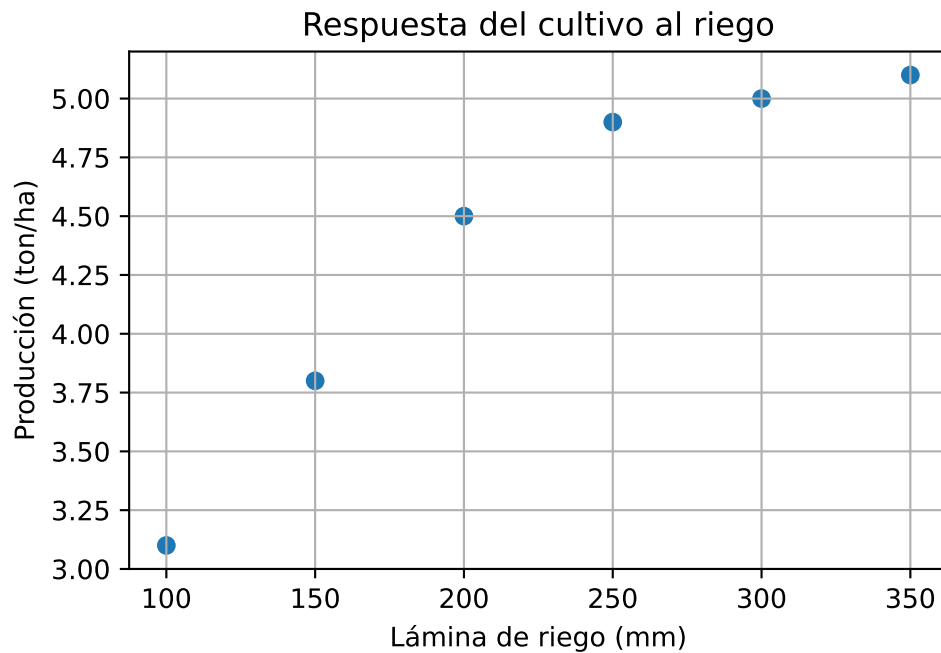


5.5.1. Otros tipos de gráfica útiles

Dispersión (scatter): para ver relación entre dos variables, por ejemplo inversión en riego vs. producción:

```
riego = np.array([100, 150, 200, 250, 300, 350]) # mm de agua
produccion = np.array([3.1, 3.8, 4.5, 4.9, 5.0, 5.1]) # ton/ha

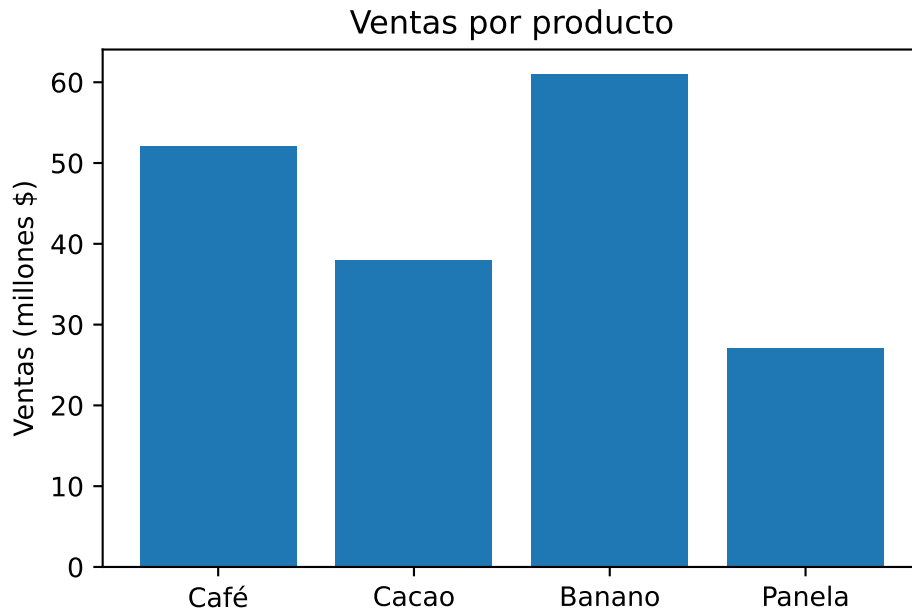
plt.scatter(riego, produccion)
plt.xlabel("Lámina de riego (mm)")
plt.ylabel("Producción (ton/ha)")
plt.title("Respuesta del cultivo al riego")
plt.grid(True)
plt.show()
```



Barras (bar): para comparar categorías:

```
productos = ["Café", "Cacao", "Banano", "Panela"]
ventas = [52, 38, 61, 27] # millones

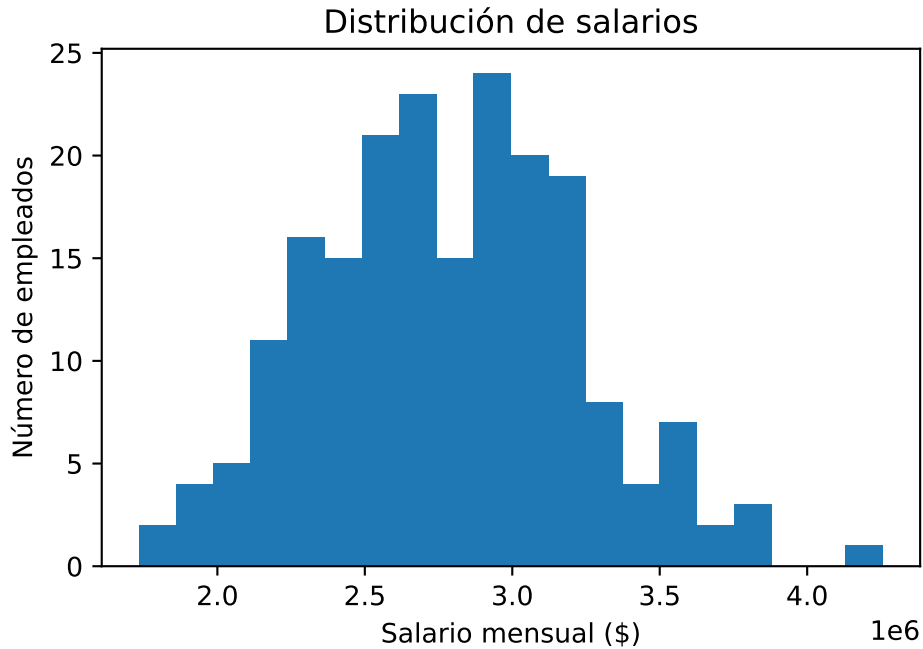
plt.bar(productos, ventas)
plt.ylabel("Ventas (millones $)")
plt.title("Ventas por producto")
plt.show()
```



Histograma (hist): para ver cómo se distribuyen los datos:

```
rng = np.random.default_rng(42)
salarios = rng.normal(2800000, 500000, 200) # 200 salarios simulados

plt.hist(salarios, bins=20)
plt.xlabel("Salario mensual ($)")
plt.ylabel("Número de empleados")
plt.title("Distribución de salarios")
plt.show()
```



5.6. Aplicación de ingeniería: simular y graficar un fenómeno

El flujo de trabajo ingenieril completo: **modelo matemático** → **array de tiempo** → **simulación** → **gráfica**. Ejemplo: la altura de una planta de maíz sigue aproximadamente una curva logística

$$h(t) = \frac{H}{1 + e^{-k(t-t_0)}}$$

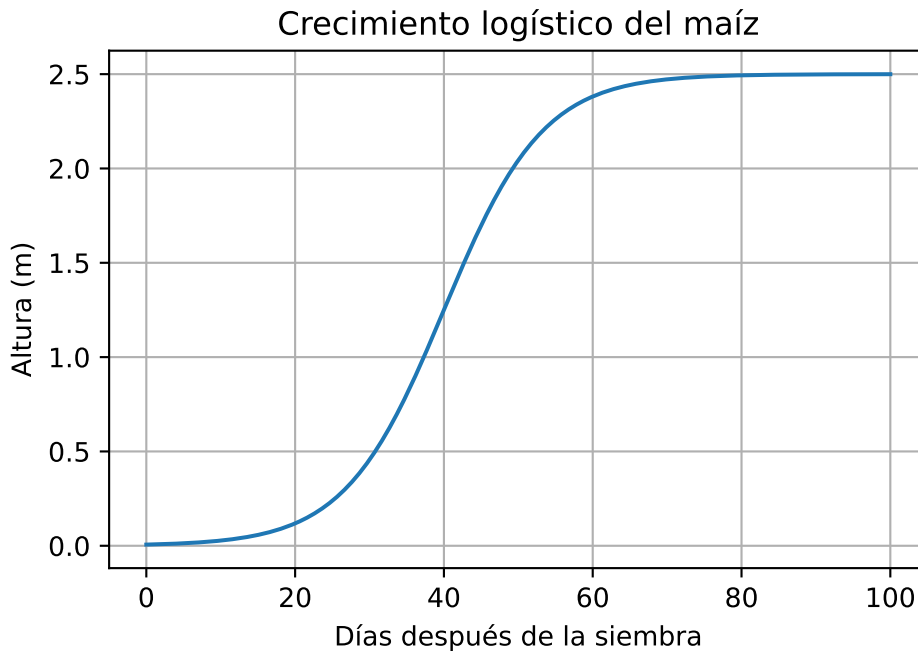
donde H es la altura máxima, k la velocidad de crecimiento y t_0 el día de crecimiento más rápido:

```
H = 2.5      # altura máxima (m)
k = 0.15     # velocidad de crecimiento
t0 = 40      # día de máximo crecimiento

t = np.linspace(0, 100, 200)      # 100 días, 200 puntos
h = H / (1 + np.exp(-k * (t - t0)))

plt.plot(t, h)
plt.xlabel("Días después de la siembra")
```

```
plt.ylabel("Altura (m)")
plt.title("Crecimiento logístico del maíz")
plt.grid(True)
plt.show()
```



Cambia k a 0.25 y vuelve a ejecutar: verás cómo el modelo responde. Este ciclo *modelo* $\rightarrow$ *simulación* $\rightarrow$ *gráfica* es exactamente el que usarás en proyectos reales de ingeniería.

5.7. Ejercicios

1. Convierte un array de precios en dólares a pesos colombianos (tasa: 1 USD = 4200 COP) usando vectorización: `precios_usd = np.array([10, 25, 7.5, 100])`.
2. Genera 50 puntos entre 0 y 2π con `linspace` y grafica $\sin(x)$ con etiquetas y grilla.
3. Simula la temperatura de un motor que se calienta según $T(t) = 25 + 65(1 - e^{-t/10})$ °C, para t entre 0 y 60 minutos. Grafica y determina a qué valor tiende la temperatura.
4. Con el array `costos = np.array([4.5, 2.1, 8.3, 1.9, 6.7, 3.2])` (millones), encuentra con `argmax()` el índice del costo mayor y su valor.
5. Grafica con barras las ventas de 6 meses que tú inventes, con título y etiquetas.
6. **Mini-reto (mecatrónica + administración):** un sensor de humedad da lecturas crudas en voltios: `v = np.array([0.8, 1.2, 1.7, 2.1, 2.9, 3.3])`. La calibración del fabricante es $\text{humedad} = 18 \cdot v - 5$ (%). (a) Convierte el array completo a humedad.

(b) Grafica voltaje vs. humedad. (c) La empresa solo vende el sensor si la humedad máxima medida supera el 50 %; imprime con un `if` si pasa o no la prueba.

5.8. Resumen

- **NumPy** trabaja con **arrays**: operaciones sobre todos los datos a la vez (**vectorización**), sin ciclos.
- `arange`, `linspace`, `zeros`, `ones` generan datos; `.mean()`, `.std()`, `.max()`, `.argmax()` los resumen.
- **Matplotlib** grafica: `plot` (líneas), `scatter` (dispersión), `bar` (barras), `hist` (histogramas), siempre con `xlabel`, `ylabel`, `title`, `grid`.
- El flujo **modelo** → **array de tiempo** → **simulación** → **gráfica** es la base del cómputo en ingeniería.

6 pandas: manejo de datos

NumPy maneja números en arreglos. Pero los datos reales vienen en **tablas**: filas y columnas con nombres, como una hoja de Excel — ventas por mes, lecturas de sensores, fichas de empleados. La librería que domina las tablas en Python es **pandas**. Este capítulo es el corazón del análisis de datos.

```
import pandas as pd
```

La convención universal es el apodo `pd`. En Colab ya viene instalada.

6.1. El DataFrame: una tabla con superpoderes

El objeto central de pandas es el **DataFrame**. Puedes crearlo desde un diccionario (¿recuerdas las fichas del capítulo 4?):

```
import pandas as pd

ventas = pd.DataFrame({
    "producto": ["café", "cacao", "banano", "café", "cacao", "banano"],
    "mes":      ["enero", "enero", "enero", "febrero", "febrero", "febrero"],
    "cantidad": [120, 80, 200, 135, 75, 220],
    "precio":   [9500, 8200, 7600, 9600, 8200, 7400]
})

print(ventas)
```

	producto	mes	cantidad	precio
0	café	enero	120	9500
1	cacao	enero	80	8200
2	banano	enero	200	7600
3	café	febrero	135	9600
4	cacao	febrero	75	8200
5	banano	febrero	220	7400

Cada columna es una **Serie** (como un array con nombre). Cada fila es un registro. Fíjate que pandas imprime la tabla con su índice a la izquierda.

6.2. Inspeccionar los datos

Lo primero que hace un analista con una tabla nueva: mirarla por encima.

```
print(ventas.head(3))      # primeras 3 filas
```

	producto	mes	cantidad	precio
0	café	enero	120	9500
1	cacao	enero	80	8200
2	banano	enero	200	7600

```
print(ventas.describe())   # estadísticas de las columnas numéricas
```

	cantidad	precio
count	6.000000	6.000000
mean	138.333333	8416.666667
std	60.387637	934.701378
min	75.000000	7400.000000
25%	90.000000	7750.000000
50%	127.500000	8200.000000
75%	183.750000	9175.000000
max	220.000000	9600.000000

```
print(ventas.info())       # tipos de datos y valores faltantes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   producto    6 non-null     object
1   mes         6 non-null     object
2   cantidad    6 non-null     int64
3   precio      6 non-null     int64
dtypes: int64(2), object(2)
memory usage: 324.0+ bytes
None
```

6.3. Seleccionar y filtrar

Seleccionar una columna (el resultado es una Serie):

```
print(ventas["producto"])
```

```
0    café
1    cacao
2    banano
3    café
4    cacao
5    banano
Name: producto, dtype: object
```

Filtrar filas con una condición — esto se llama **indexación booleana** y es de lo más usado en pandas:

```
ventas_grandes = ventas[ventas["cantidad"] > 150]
print(ventas_grandes)
```

	producto	mes	cantidad	precio
2	banano	enero	200	7600
5	banano	febrero	220	7400

Se lee así: “de la tabla `ventas`, dame las filas donde `cantidad > 150`”. La condición produce un True/False por fila, y pandas se queda con las verdaderas.

6.4. Columnas calculadas

Como en NumPy, las operaciones son vectorizadas: se aplican a toda la columna de una vez. Crear una columna nueva es asignarla:

```
ventas["ingreso"] = ventas["cantidad"] * ventas["precio"]
print(ventas)
```

	producto	mes	cantidad	precio	ingreso
0	café	enero	120	9500	1140000
1	cacao	enero	80	8200	656000
2	banano	enero	200	7600	1520000
3	café	febrero	135	9600	1296000

4	cacao	febrero	75	8200	615000
5	banano	febrero	220	7400	1628000

6.5. Agrupar: la pregunta de negocio típica

“¿Cuánto vendimos por producto?” es la pregunta clásica de administración. Con `groupby` se responde en una línea:

```
por_producto = ventas.groupby("producto")["ingreso"].sum()
print(por_producto)
```

```
producto
banano    3148000
cacao     1271000
café      2436000
Name: ingreso, dtype: int64
```

`groupby("producto")` junta las filas del mismo producto; `["ingreso"].sum()` suma el ingreso dentro de cada grupo. También puedes usar `.mean()`, `.count()`, `.max()`.

Y ordenar para ver al ganador primero:

```
print(por_producto.sort_values(ascending=False))
```

```
producto
banano    3148000
café      2436000
cacao     1271000
Name: ingreso, dtype: int64
```

6.6. Leer datos reales: `read_csv`

Los datos rara vez se escriben a mano: vienen en archivos **CSV** (valores separados por comas). pandas los lee con una línea — incluso desde una URL, ideal para Colab:

```
url = "https://raw.githubusercontent.com/plotly/datasets/master/iris.csv"
iris = pd.read_csv(url)
print(iris.head())
```

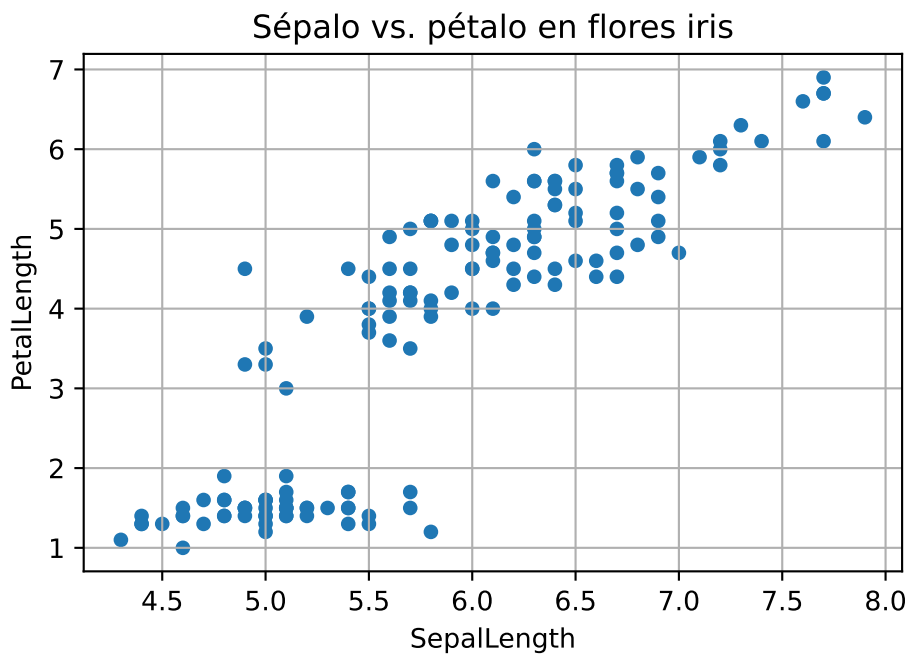
	SepalLength	SepalWidth	PetalLength	PetalWidth	Name
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Ese dataset contiene medidas de flores *iris* (largo y ancho de sépalo y pétalo). Ya tienes datos reales en tu programa. En Colab también puedes subir un archivo y leerlo con `pd.read_csv("archivo.csv")`.

Con pandas, la gráfica de un DataFrame es inmediata:

```
import matplotlib.pyplot as plt

iris.plot(kind="scatter", x="SepalLength", y="PetalLength")
plt.title("Sépalo vs. pétalo en flores iris")
plt.grid(True)
plt.show()
```



6.7. Series de tiempo (primera mirada)

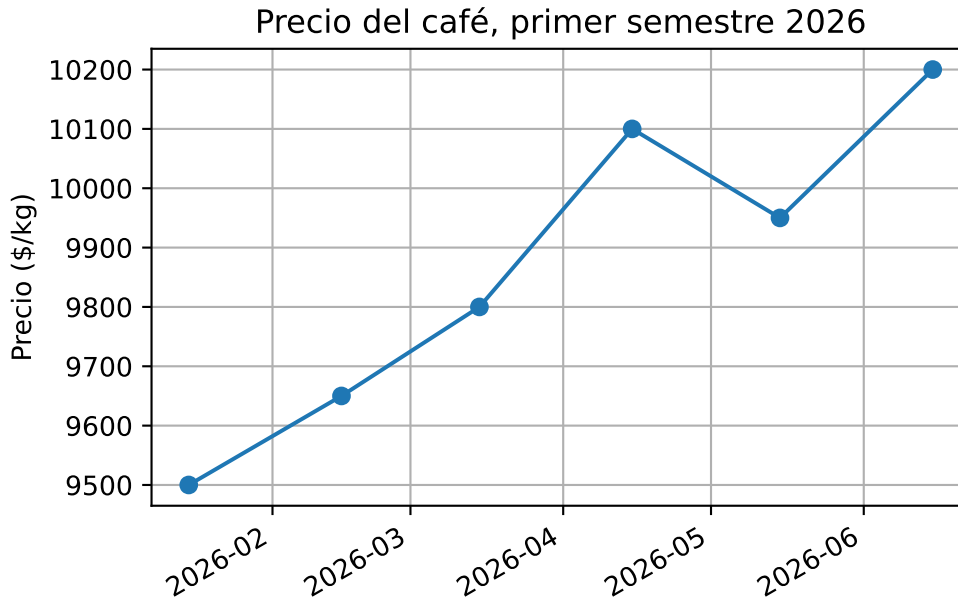
Las fechas son datos especiales. pandas las entiende si se las presentas bien:

```
fechas = pd.to_datetime(["2026-01-15", "2026-02-15", "2026-03-15",  
                        "2026-04-15", "2026-05-15", "2026-06-15"])  
precio_cafe = pd.Series([9500, 9650, 9800, 10100, 9950, 10200], index=fechas)  
  
print(precio_cafe)
```

```
2026-01-15    9500  
2026-02-15    9650  
2026-03-15    9800  
2026-04-15   10100  
2026-05-15    9950  
2026-06-15   10200  
dtype: int64
```

Con la fecha como índice, graficar la serie es trivial:

```
precio_cafe.plot(marker="o")  
plt.ylabel("Precio ($/kg)")  
plt.title("Precio del café, primer semestre 2026")  
plt.grid(True)  
plt.show()
```



En el capítulo 7 usaremos esto para análisis económicos más elaborados.

6.8. Ejercicios

1. Crea un DataFrame con 5 empleados (nombre, cargo, salario) y calcula la columna `salario_anual`.
2. Del DataFrame `ventas` de este capítulo, filtra las filas donde el ingreso supere \$1.500.000.
3. Agrupa `ventas` por `mes` y calcula el ingreso total de cada mes. ¿Cuál mes fue mejor?
4. Lee el dataset iris desde la URL y calcula el promedio de `PetalLength` con `.mean()`.
5. Del dataset iris, filtra las flores con `SepalLength` mayor a 6.5 y cuenta cuántas son con `len()`.
6. **Mini-reto (agro):** construye un DataFrame con las lecturas horarias simuladas de un sensor: columnas `hora` (0 a 23), `temperatura` (valores entre 18 y 34) y `humedad` (valores entre 40 y 90). (a) Filtra las horas con temperatura mayor a 30. (b) Calcula el promedio de humedad. (c) Crea la columna `alerta` que valga `True` cuando `temperatura > 30` y `humedad < 50` (pista: `(df["temperatura"] > 30) & (df["humedad"] < 50)`). (d) Grafica temperatura vs. hora.

6.9. Resumen

- **pandas** maneja tablas: el **DataFrame** (tabla) y la **Serie** (columna).
- `head()`, `describe()`, `info()` inspeccionan; `df["col"]` selecciona; `df[condicion]` filtra.

- Las columnas calculadas son vectorizadas: `df["nueva"] = df["a"] * df["b"]`.
- `groupby("col").sum()` responde preguntas de negocio por categorías.
- `pd.read_csv()` trae datos reales desde archivos o URLs.
- Con fechas como índice, las series de tiempo se grafican directo.

7 Aplicaciones en administración y economía

Llegó el momento de poner todo a trabajar: variables, funciones, ciclos, NumPy, pandas y gráficas. En este capítulo resolverás cuatro problemas clásicos de administración y economía: **interés compuesto**, **valor actual neto (VAN)**, **tasa interna de retorno (TIR)** y **simulación Monte Carlo** para decisiones bajo incertidumbre.

7.1. Valor futuro y valor presente

El dinero tiene fecha: \$1.000.000 hoy no valen lo mismo que \$1.000.000 en cinco años. El **valor futuro** ya lo conoces:

```
def valor_futuro(capital, tasa, años):  
    return capital * (1 + tasa) ** años  
  
vf = valor_futuro(1000000, 0.12, 5)  
print(f"Valor futuro: ${vf:,.0f}")
```

```
Valor futuro: $1,762,342
```

La operación inversa es el **valor presente**: ¿cuánto vale hoy un pago futuro?

```
def valor_presente(pago_futuro, tasa, años):  
    return pago_futuro / (1 + tasa) ** años  
  
vp = valor_presente(1762342, 0.12, 5)  
print(f"Valor presente: ${vp:,.0f}")
```

```
Valor presente: $1,000,000
```

7.2. Valor Actual Neto (VAN): ¿vale la pena el proyecto?

Un proyecto es una secuencia de **flujos de caja**: primero inviertes (flujo negativo) y luego recibes ingresos año a año. El VAN trae todos esos flujos a valor presente y los suma:

$$VAN = \sum_{t=0}^n \frac{F_t}{(1+r)^t}$$

Regla de decisión: $VAN > 0 \rightarrow$ el proyecto crea valor; $VAN < 0 \rightarrow$ lo destruye.

```
def van(tasa, flujos):
    return sum(f / (1 + tasa) ** t for t, f in enumerate(flujos))

# Proyecto: invertir 10 millones hoy, recibir 3 millones por 5 años
flujos = [-10000000, 3000000, 3000000, 3000000, 3000000, 3000000]
tasa_descuento = 0.12

resultado = van(tasa_descuento, flujos)
print(f"VAN al 12%: ${resultado:,.0f}")

if resultado > 0:
    print("El proyecto crea valor: adelante")
else:
    print("El proyecto destruye valor: no invertir")
```

```
VAN al 12%: $814,329
El proyecto crea valor: adelante
```

La función usa dos cosas nuevas que vale la pena mirar: `enumerate(flujos)` entrega cada flujo con su posición `t` (el año), y la construcción `sum(... for ... in ...)` es una **expresión generadora**: suma sin necesidad de crear una lista intermedia.

7.3. Tasa Interna de Retorno (TIR)

La TIR es la tasa que hace el VAN exactamente cero: la rentabilidad “verdadera” del proyecto. No tiene fórmula directa; se encuentra por **búsqueda numérica**. El método de **bisección** parte el intervalo a la mitad en cada paso:

```
def tir(flujos):
    bajo, alto = 0.0, 1.0          # la TIR está entre 0% y 100%
    for _ in range(100):          # 100 iteraciones son de sobra
        medio = (bajo + alto) / 2
        if van(medio, flujos) > 0: # si el VAN es positivo, la TIR está arriba
            bajo = medio
        else:
            alto = medio
    return medio

tasa_interna = tir(flujos)
print(f"TIR del proyecto: {tasa_interna:.2%}")
```

```
TIR del proyecto: 15.24%
```

Observa el formato `{tasa_interna:.2%}`: convierte el decimal a porcentaje con dos cifras. Como la TIR (15,24%) supera la tasa de descuento (12%), el proyecto es atractivo — la misma conclusión del VAN, vista desde la rentabilidad.

7.4. Regresión lineal: aprender de los datos

La regresión encuentra la recta que mejor describe la relación entre dos variables: $y = mx + b$. Con NumPy se calcula en una línea:

```
import numpy as np
import matplotlib.pyplot as plt

publicidad = np.array([1, 2, 3, 4, 5, 6])          # millones invertidos
ventas = np.array([52, 58, 65, 71, 79, 85])        # millones vendidos

m, b = np.polyfit(publicidad, ventas, 1)           # pendiente y corte
print(f"Modelo: ventas = {m:.1f} × publicidad + {b:.1f}")
```

```
Modelo: ventas = 6.7 × publicidad + 44.9
```

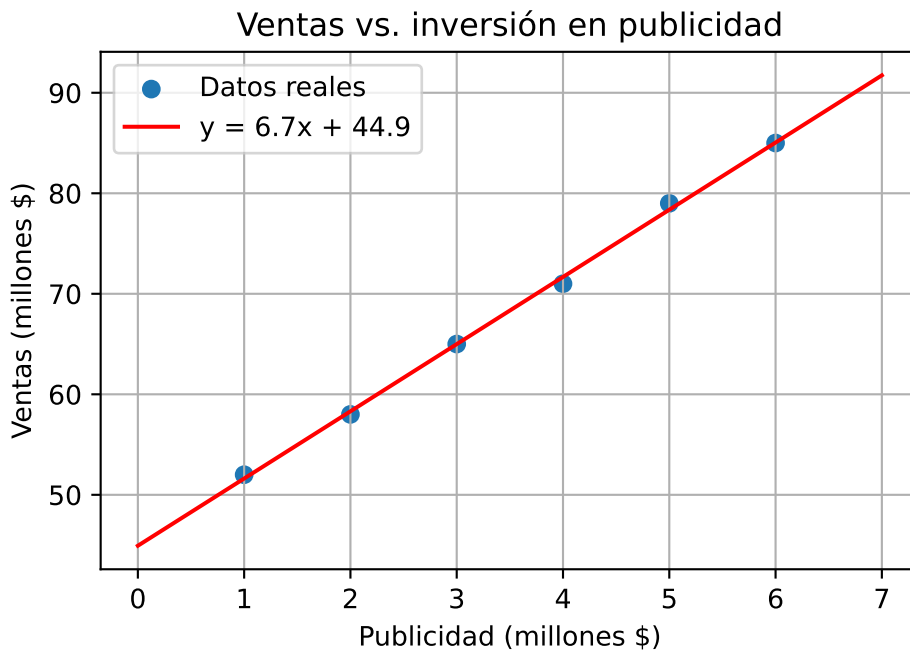
Cada millón adicional en publicidad se asocia con `m` millones adicionales en ventas. Visualicemos datos y modelo:

```

x = np.linspace(0, 7, 100)

plt.scatter(publicidad, ventas, label="Datos reales")
plt.plot(x, m * x + b, color="red", label=f"y = {m:.1f}x + {b:.1f}")
plt.xlabel("Publicidad (millones $)")
plt.ylabel("Ventas (millones $)")
plt.title("Ventas vs. inversión en publicidad")
plt.legend()
plt.grid(True)
plt.show()

```



Con el modelo puedes **predecir**: si inviertes 8 millones, esperas $m \cdot 8 + b$ millones en ventas. (Ojo: predecir dentro del rango observado es razonable; extrapolar muy lejos es peligroso).

7.5. Simulación Monte Carlo: decidir con incertidumbre

La vida real no tiene un solo futuro, tiene miles. La simulación Monte Carlo genera muchos escenarios aleatorios y mira qué pasa “en promedio” y “en el peor caso”.

Ejemplo: la utilidad de una cosecha depende del precio y del rendimiento, ambos inciertos. Supón que el precio sigue una distribución normal con media \$9.500/kg y desviación \$800, y el rendimiento es normal con media 4,2 ton/ha y desviación 0,6:

```

rng = np.random.default_rng(42)

escenarios = 10000
precios = rng.normal(9500, 800, escenarios)      # $/kg
rendimientos = rng.normal(4.2, 0.6, escenarios)   # ton/ha

hectareas = 10
costos = 28000000

utilidad = precios * rendimientos * 1000 * hectareas - costos

print(f"Utilidad esperada: ${utilidad.mean():,.0f}")
print(f"Peor 5% de escenarios: ${np.percentile(utilidad, 5):,.0f}")
print(f"Probabilidad de pérdida: {(utilidad < 0).mean():.1%}")

```

```

Utilidad esperada: $371,754,211
Peor 5% de escenarios: $267,681,774
Probabilidad de pérdida: 0.0%

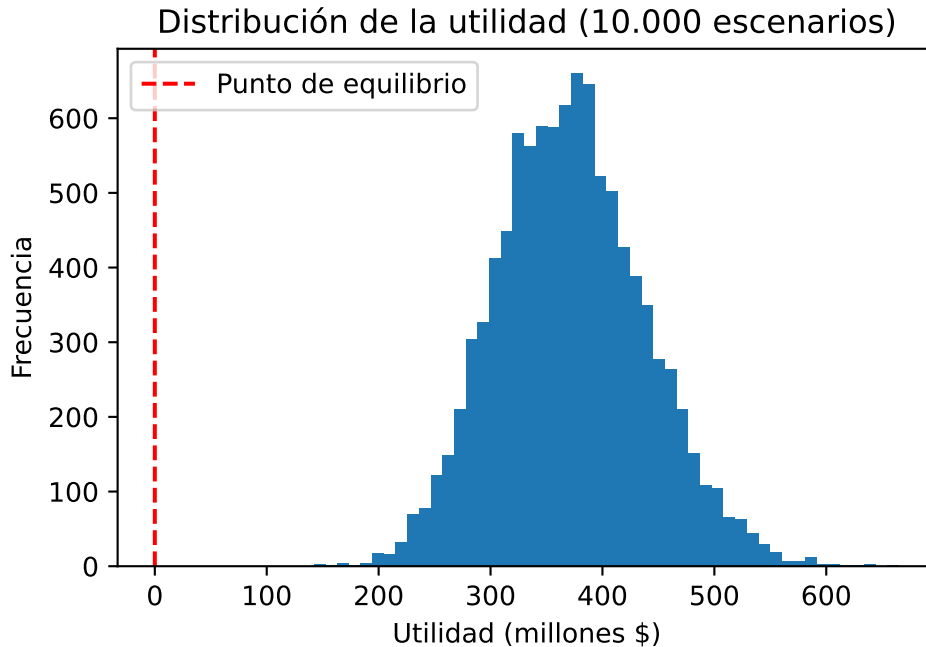
```

Y la gráfica que resume todo el riesgo:

```

plt.hist(utilidad / 1e6, bins=50)
plt.axvline(0, color="red", linestyle="--", label="Punto de equilibrio")
plt.xlabel("Utilidad (millones $)")
plt.ylabel("Frecuencia")
plt.title("Distribución de la utilidad (10.000 escenarios)")
plt.legend()
plt.show()

```



En vez de un solo número, ahora tienes la **distribución completa**: la utilidad más probable, el peor escenario razonable (percentil 5) y la probabilidad exacta de perder plata. Así deciden los analistas modernos.

7.6. Ejercicios

1. Calcula el valor presente de recibir \$20.000.000 en 3 años con tasa del 10 %.
2. Evalúa con VAN (tasa 15 %) este proyecto: inversión de \$50 millones hoy, flujos de \$15, \$18, \$20, \$22 y \$25 millones en los años 1 a 5. ¿Se acepta?
3. Calcula la TIR del proyecto del ejercicio 2 con la función `tir()` de este capítulo.
4. Ajusta una regresión entre estos datos de fertilizante (kg/ha) y rendimiento (ton/ha):
`fert = np.array([50, 80, 110, 140, 170])`, `rend = np.array([3.1, 3.9, 4.6, 5.0, 5.2])`. Predice el rendimiento para 120 kg/ha y grafica datos y recta.
5. Replica la simulación Monte Carlo de la cosecha pero con 20 hectáreas y costos de \$55 millones. ¿Sube o baja la probabilidad de pérdida?
6. **Mini-reto (decisión de inversión completa)**: una empresa evalúa comprar una planta empacadora por \$80 millones. Los flujos esperados son \$20 millones anuales por 6 años, la tasa de descuento es 14 %. (a) Calcula el VAN y decide. (b) Calcula la TIR. (c) Los flujos son inciertos: simula 10.000 escenarios donde cada flujo anual es normal con media 20 y desviación 4 (millones), calcula el VAN de cada escenario (usa un `for` sobre escenarios) y reporta la probabilidad de que el proyecto destruya valor. (d) Grafica el histograma de los VAN simulados.

7.7. Resumen

- **Valor futuro/presente:** el dinero se mueve en el tiempo con $(1 + r)^n$.
- **VAN:** suma de flujos descontados; positivo $\rightarrow$ invertir. Se implementa con `sum` y `enumerate`.
- **TIR:** la tasa que hace $VAN = 0$; se encuentra por búsqueda (bisección).
- **Regresión lineal** con `np.polyfit`: convierte datos en un modelo predictivo $y = mx + b$.
- **Monte Carlo:** miles de escenarios aleatorios revelan la utilidad esperada, el peor caso y la probabilidad de pérdida.

8 Aplicaciones en ingeniería: agro, mecatrónica, veterinaria y zootecnia

Python es hoy una herramienta de trabajo diaria en las ingenierías. En este capítulo aplicarás lo aprendido a cuatro problemas reales: **balance hídrico para riego** (ingeniería agrícola), **filtrado de señales de sensores** (mecatrónica), **dosificación de medicamentos** (veterinaria) y **crecimiento animal** (zootecnia). El patrón siempre es el mismo: modelo matemático → código vectorizado → gráfica → decisión.

8.1. Ingeniería agrícola: ¿cuánta agua necesita el cultivo?

El riego se programa con un **balance hídrico**: el cultivo pierde agua por evapotranspiración (ET_c) y la lluvia repone parte; la diferencia es la lámina de riego que debes aplicar:

$$L = ET_c - P_{ef} = K_c \cdot ET_0 - P_{ef}$$

donde K_c es el coeficiente del cultivo y P_{ef} la lluvia efectiva. Un método sencillo para estimar ET_0 con datos de temperatura (Hargreaves simplificado):

```
import numpy as np
import matplotlib.pyplot as plt

# Semana típica: temperaturas máximas y mínimas (°C) y lluvia (mm)
t_max = np.array([32, 33, 31, 34, 33, 32, 31])
t_min = np.array([20, 21, 19, 22, 21, 20, 19])
lluvia = np.array([0, 5, 0, 12, 0, 0, 2])

et0 = 0.0135 * ((t_max + t_min) / 2 + 17.8) * np.sqrt(t_max - t_min) * 0.16

kc = 1.05          # maíz en etapa media
etc = kc * et0
lamina = np.maximum(etc - lluvia, 0)  # nunca negativa

print("ET0 diaria (mm):", np.round(et0, 2))
```

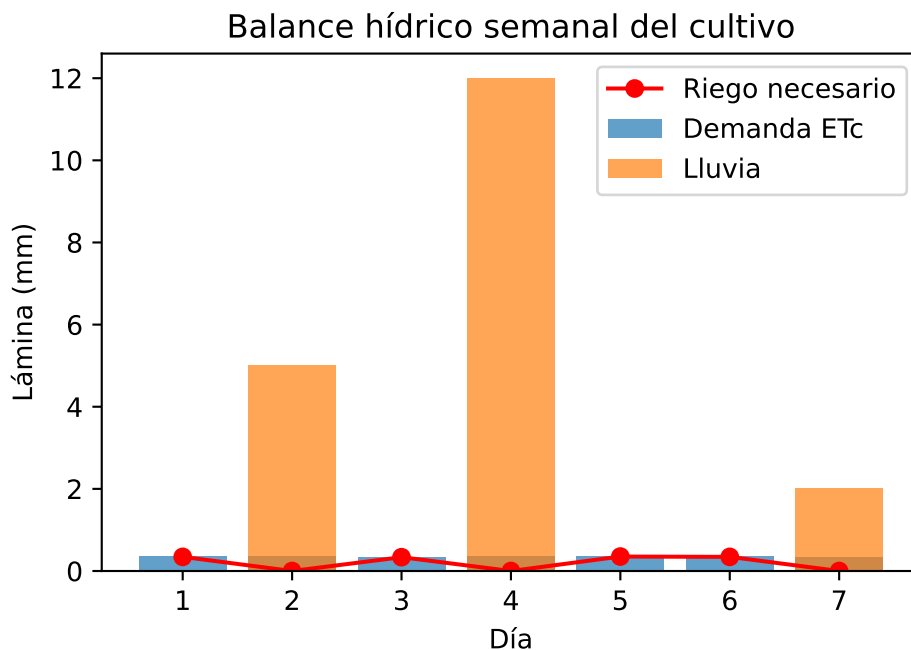
```
print("Lámina de riego diaria (mm):", np.round(lamina, 2))
print(f"Total semanal: {lamina.sum():.1f} mm = {lamina.sum() * 10:.0f} m³/ha")
```

```
ET0 diaria (mm): [0.33 0.34 0.32 0.34 0.34 0.33 0.32]
Lámina de riego diaria (mm): [0.34 0.    0.34 0.    0.35 0.34 0.   ]
Total semanal: 1.4 mm = 14 m³/ha
```

`np.maximum(..., 0)` aplica el límite elemento por elemento: si la lluvia supera la demanda, no se riega. El factor final convierte mm a m³/ha (1 mm sobre 1 ha = 10 m³). Grafiquemos el balance:

```
dias = np.arange(1, 8)

plt.bar(dias, etc, label="Demanda ETc", alpha=0.7)
plt.bar(dias, lluvia, label="Lluvia", alpha=0.7)
plt.plot(dias, lamina, "ro-", label="Riego necesario")
plt.xlabel("Día")
plt.ylabel("Lámina (mm)")
plt.title("Balance hídrico semanal del cultivo")
plt.legend()
plt.show()
```



8.2. Mecatrónica: limpiar la señal de un sensor

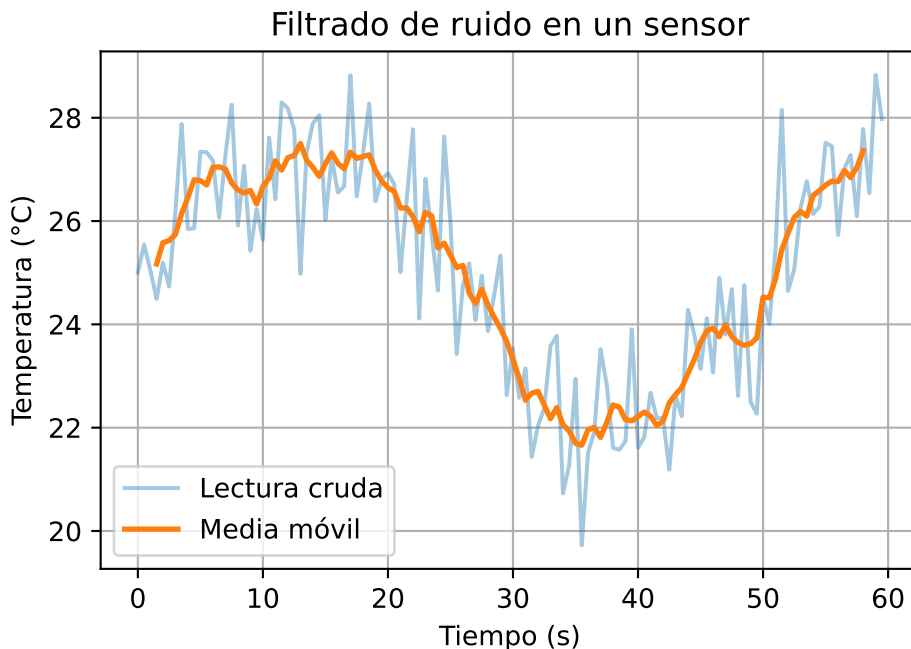
Los sensores reales entregan señales con **ruido**. Un filtro clásico y simple es la **media móvil**: cada valor se reemplaza por el promedio de sus vecinos, y el ruido se suaviza:

```
import pandas as pd

rng = np.random.default_rng(7)
tiempo = np.arange(0, 60, 0.5)                # 2 lecturas por segundo
senal_real = 25 + 3 * np.sin(tiempo / 8)       # temperatura verdadera
lectura = senal_real + rng.normal(0, 1.2, len(tiempo)) # + ruido del sensor

df = pd.DataFrame({"tiempo": tiempo, "lectura": lectura})
df["filtrada"] = df["lectura"].rolling(window=7, center=True).mean()

plt.plot(df["tiempo"], df["lectura"], alpha=0.4, label="Lectura cruda")
plt.plot(df["tiempo"], df["filtrada"], linewidth=2, label="Media móvil")
plt.xlabel("Tiempo (s)")
plt.ylabel("Temperatura (°C)")
plt.title("Filtrado de ruido en un sensor")
plt.legend()
plt.grid(True)
plt.show()
```



`rolling(window=7)` crea una ventana de 7 puntos; `.mean()` la promedia. Luego, la lógica de control: detectar cuándo la señal **filtrada** cruza el umbral (con la cruda habría falsas alarmas):

```
umbral = 27.5
alarma = df["filtrada"] > umbral
segundos_en_alarma = alarma.sum() * 0.5 # 0.5 s por lectura
print(f"Tiempo sobre el umbral: {segundos_en_alarma:.1f} s")
```

```
Tiempo sobre el umbral: 0.5 s
```

8.3. Veterinaria: dosificación por peso vivo

La dosis de muchos medicamentos se calcula por kilogramo de peso vivo. Con un lote de animales, la vectorización evita errores de cálculo manual:

```
animales = pd.DataFrame({
    "id": ["A01", "A02", "A03", "A04", "A05"],
    "peso_kg": [380, 425, 350, 410, 365]
})

dosis_mg_kg = 10 # mg por kg de peso vivo
concentracion = 200 # mg por ml del producto comercial

animales["dosis_mg"] = animales["peso_kg"] * dosis_mg_kg
animales["ml_aplicar"] = animales["dosis_mg"] / concentracion

print(animales)
print(f"\nTotal de producto para el lote: {animales['ml_aplicar'].sum():.1f} ml")
```

	id	peso_kg	dosis_mg	ml_aplicar
0	A01	380	3800	19.00
1	A02	425	4250	21.25
2	A03	350	3500	17.50
3	A04	410	4100	20.50
4	A05	365	3650	18.25

Total de producto para el lote: 96.5 ml

Una tabla clara por animal y el total de producto a comprar: eso es exactamente lo que necesita un plan sanitario.

8.4. Zootecnia: curva de crecimiento y ganancia diaria

El peso de un animal en crecimiento sigue bien el modelo de **Gompertz**:

$$W(t) = A e^{-b e^{-kt}}$$

donde A es el peso adulto, k la velocidad de maduración y b un parámetro de forma. Simulemos un lote de pollos de engorde:

```
A, b, k = 4500, 4.5, 0.045    # peso adulto (g), forma, velocidad
t = np.linspace(0, 100, 200) # días

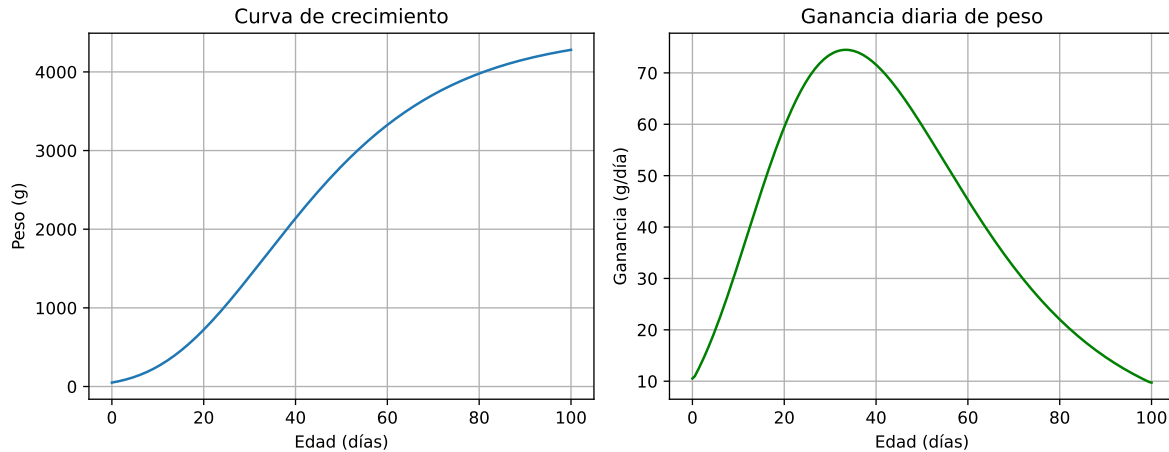
peso = A * np.exp(-b * np.exp(-k * t))
ganancia = np.gradient(peso, t)    # derivada numérica: g/día

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

ax1.plot(t, peso)
ax1.set_xlabel("Edad (días)")
ax1.set_ylabel("Peso (g)")
ax1.set_title("Curva de crecimiento")
ax1.grid(True)

ax2.plot(t, ganancia, color="green")
ax2.set_xlabel("Edad (días)")
ax2.set_ylabel("Ganancia (g/día)")
ax2.set_title("Ganancia diaria de peso")
ax2.grid(True)

plt.tight_layout()
plt.show()
```



Dos novedades: `np.gradient` calcula la **derivada numérica** (la ganancia diaria sin derivar a mano), y `plt.subplots(1, 2)` pone dos gráficas lado a lado. El día de máxima ganancia marca el momento económico óptimo de venta:

```
dia_optimo = t[np.argmax(ganancia)]
print(f"Máxima ganancia diaria: {ganancia.max():.1f} g/día, al día {dia_optimo:.0f}")
```

```
Máxima ganancia diaria: 74.5 g/día, al día 34
```

8.5. Ejercicios

1. Recalcula el balance hídrico con $K_c = 0.75$ (etapa inicial del cultivo). ¿Cuánto baja la lámina semanal?
2. En la señal del sensor, cambia la ventana de la media móvil a 3 y luego a 15. ¿Qué ventana filtra mejor sin deformar la señal? Justifica con las gráficas.
3. Agrega al DataFrame de animales una columna `costo` si el producto vale \$850 por ml. ¿Cuánto cuesta tratar todo el lote?
4. Simula con Gompertz el crecimiento de terneros: $A = 480$ kg, $b = 3.8$, $k = 0.0035$, hasta 800 días. Encuentra el día de máxima ganancia.
5. Calcula la **conversión alimenticia** de un lote: consumo total de alimento 8.2 toneladas, ganancia total de peso 2.5 toneladas. Luego, si el alimento cuesta \$1.800/kg y el kilo en pie se vende a \$8.500, ¿cuál fue el margen por kilo producido?
6. **Mini-reto integrador (agro + mecatrónica + economía)**: un sistema de riego registra la humedad del suelo cada hora durante 5 días. (a) Simula la serie: humedad base que baja 2 puntos por día desde 55%, más ruido normal de desviación 1.5 (usa `rng.normal`). (b) Filtra con media móvil de ventana 5. (c) Detecta en qué hora la señal

filtrada cruza por debajo del 30 % (umbral de riego) — pista: `np.argmax(filtrada < 30)`. (d) Grafica cruda + filtrada + línea horizontal del umbral. (e) Cada riego cuesta \$180.000; si el sistema riega cada vez que se cruza el umbral y la humedad se recupera a 55 % tras cada riego, estima cuántos riegos hubo en la simulación (puedes implementar el reinicio con un `while`).

8.6. Resumen

- El patrón ingenieril: **modelo** → **vectorización** → **gráfica** → **decisión**.
- Agro: balance hídrico $L = K_c \cdot ET_0 - P_{ef}$ para programar riego.
- Mecatrónica: la media móvil (**rolling**) filtra ruido antes de aplicar umbrales de control.
- Veterinaria: dosificación por peso con columnas calculadas en pandas.
- Zootecnia: modelo de Gompertz + `np.gradient` para ganancia diaria y momento óptimo de venta.

9 Proyectos integradores

Cada capítulo terminó con ejercicios guiados. Ahora es distinto: aquí hay **cinco proyectos abiertos**, uno por disciplina. En todos ellos pasa lo mismo que en la vida real: el enunciado no trae los datos ni todas las fórmulas — **tendrás que investigar** (fuentes oficiales, fichas técnicas, literatura) — y las preguntas son imposibles de responder a mano: **la computación es indispensable**.

Reglas del juego para todos los proyectos:

- Todo el análisis en un notebook de Colab, con celdas de texto que expliquen lo que haces (usa Markdown).
- Documenta las fuentes de cada dato o fórmula que investigues (enlace + fecha de consulta).
- Toda conclusión debe apoyarse en una tabla, una gráfica o un número calculado — no en intuiciones.
- Estructura mínima del notebook: *Contexto* → *Datos y fuentes* → *Análisis* → *Gráficas* → *Conclusiones*.

9.1. Proyecto A — Ingeniería agrícola: ¿cuándo y cuánto regar?

Contexto. Eres el asistente técnico de una finca con 5 ha de maíz en el Valle del Cauca. Debes proponer un calendario de riego para un mes completo.

Lo que debes investigar:

1. Los coeficientes de cultivo K_c del maíz por etapa fenológica (fuente sugerida: FAO, documento *Crop Evapotranspiration*, FAO-56).
2. La precipitación y temperaturas históricas de tu municipio (IDEAM o NASA POWER, que tiene datos gratuitos por coordenada).
3. El método de Hargreaves completo para ET_0 (en el capítulo 8 usamos una versión simplificada — encuentra la original y compara).

Lo que debes computar:

- Una serie diaria de 30 días de ET_c , lluvia efectiva y lámina neta de riego (puedes usar datos reales o una simulación justificada con la climatología investigada).

- El volumen total de agua del mes en m^3/ha y su comparación contra el caudal disponible de la finca (dato que definirás e investigarás: caudal típico de un pozo o bocatoma pequeña).
- Gráficas del balance hídrico y una tabla-resumen por semana.
- Una recomendación final: calendario de riego (qué días, cuánta lámina) y qué pasaría con dos semanas sin lluvia.

9.2. Proyecto B — Mecatrónica: confiabilidad de un sensor de bajo costo

Contexto. Tu empresa quiere usar sensores de humedad de suelo de \$15.000 en vez de los de \$1.500.000. Antes de venderlos, debes caracterizar su comportamiento.

Lo que debes investigar:

1. La curva de calibración de un sensor capacitivo real (busca la ficha técnica del sensor *Capacitive Soil Moisture Sensor v1.2* o similar).
2. Qué es la deriva (*drift*) y el histéresis en sensores, y por qué importan en agricultura de precisión.
3. Fórmulas del filtro de media móvil exponencial (EMA) y su diferencia con la media móvil simple del capítulo 8.

Lo que debes computar:

- Simula 48 horas de lecturas (cada 5 min) de un sensor con ruido, deriva lenta y una calibración no lineal (define los parámetros con lo investigado).
- Implementa y compara media móvil simple vs. EMA; cuantifica el error de cada una respecto a la señal verdadera (RMSE — investiga su fórmula).
- Diseña una lógica de alarma con histéresis (enciende riego bajo 30 %, apaga sobre 38 %) y cuenta cuántas falsas activaciones evita respecto a un umbral simple.
- Conclusión de ingeniería: ¿el sensor barato es usable? ¿Bajo qué condiciones?

9.3. Proyecto C — Administración y economía: evaluación completa de un proyecto productivo

Contexto. Una asociación de productores evalúa montar una planta de beneficio de café. Debes entregar el análisis de viabilidad financiera.

Lo que debes investigar:

1. Precios históricos del café (Federación Nacional de Cafeteras publica el precio interno diario).
2. La tasa de descuento apropiada para proyectos rurales en Colombia (investiga qué es la tasa social de descuento del DNP).
3. Costos típicos de una planta de beneficio pequeña (literatura de extensión rural, Ceni-café).

Lo que debes computar:

- Flujo de caja a 10 años con tus supuestos documentados (inversión, capacidad, costos variables y fijos).
- VAN y TIR con las funciones del capítulo 7.
- Análisis de sensibilidad: tabla de VAN para 5 precios de café $\times$ 5 niveles de producción (usa doble ciclo `for` y un `DataFrame`).
- Monte Carlo con 10.000 escenarios (precio y producción aleatorios con distribuciones justificadas) y probabilidad de pérdida.
- Dictamen final de una página: se monta o no se monta, y por qué.

9.4. Proyecto D — Veterinaria y zootecnia: plan de ceba de un lote

Contexto. Una finca compra 40 terneros de 200 kg y quiere venderlos cuando el margen sea máximo. Los costos de alimentación suben con el peso; el precio por kg en pie es fijo.

Lo que debes investigar:

1. Parámetros reales del modelo Gompertz (o logístico) para la raza de interés (literatura zootécnica: artículos de crecimiento en *Bos indicus* o la raza local).
2. Requerimientos nutricionales y conversión alimenticia por etapa (tablas NRC o equivalentes).
3. Precios actuales del ganado en pie y del concentrado en tu región.

Lo que debes computar:

- Curva de crecimiento del lote a 700 días con los parámetros investigados.
- Costo diario de alimentación (función del peso) e ingreso potencial por venta en cada día.
- El margen por animal en función del día de venta: gráfica y día óptimo con `np.argmax`.
- Dosificación de un tratamiento para todo el lote en el día 180 (mg/kg $\rightarrow$ ml por animal, como en el capítulo 8).
- Sensibilidad: ¿cómo cambia el día óptimo si el alimento sube 20%?

9.5. Proyecto E — Ciencia de datos general: auditoría de un dataset real

Contexto. Te entregan (o tú descargas) un dataset real de tu interés — producción agrícola municipal (Datos Abiertos Colombia), precios de alimentos (DANE/SIPSA), calidad del agua — y debes auditarlo y extraerle una historia.

Lo que debes investigar:

1. Descargar un dataset real de datos.gov.co en formato CSV.
2. El diccionario de datos: qué significa cada columna.
3. Buenas prácticas de calidad de datos (valores faltantes, duplicados, outliers).

Lo que debes computar:

- Carga con `pd.read_csv` y diagnóstico: dimensiones, tipos, `describe()`, conteo de faltantes y duplicados.
- Limpieza justificada: decide qué hacer con cada problema encontrado y documéntalo.
- Tres preguntas de negocio que tú formules, respondidas con `groupby`, filtros y gráficas.
- Un hallazgo final: algo que solo se vea después del análisis.

9.6. Rúbrica sugerida (para docentes)

Criterio	Peso	Excelente (5)	Insuficiente (< 3)
Investigación y fuentes	20 %	Fuentes oficiales citadas y usadas correctamente	Sin fuentes o datos inventados sin justificar
Corrección computacional	30 %	El código corre de principio a fin sin errores en Colab	El notebook no corre
Análisis y gráficas	30 %	Gráficas con etiquetas, tablas claras, interpretación correcta	Gráficas sin contexto o mal interpretadas
Conclusiones	20 %	Decisión clara apoyada en los resultados	Conclusiones desconectadas del análisis

Tip

Para el estudiante: el objetivo no es que te salga perfecto a la primera. Es que enfrentes el ciclo completo — investigar, programar, equivocarte, corregir, concluir — que es exactamente lo que harás como profesional.

10 Capítulo 8: Introducción a big data

En construcción.